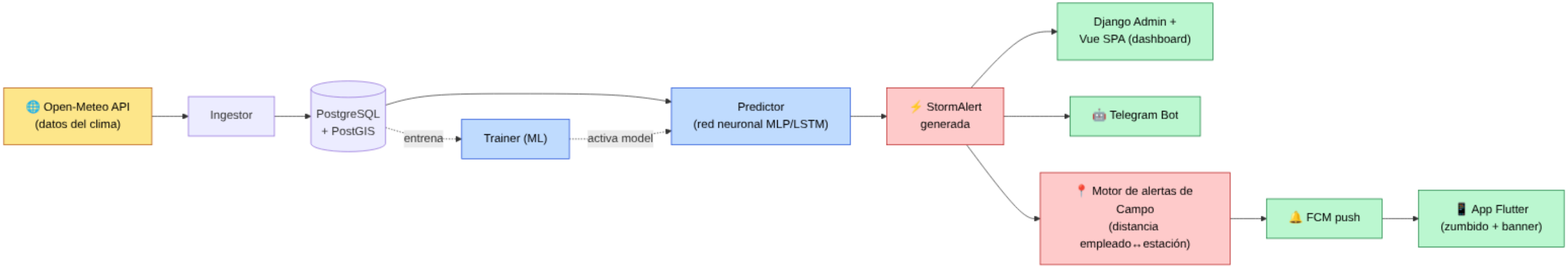
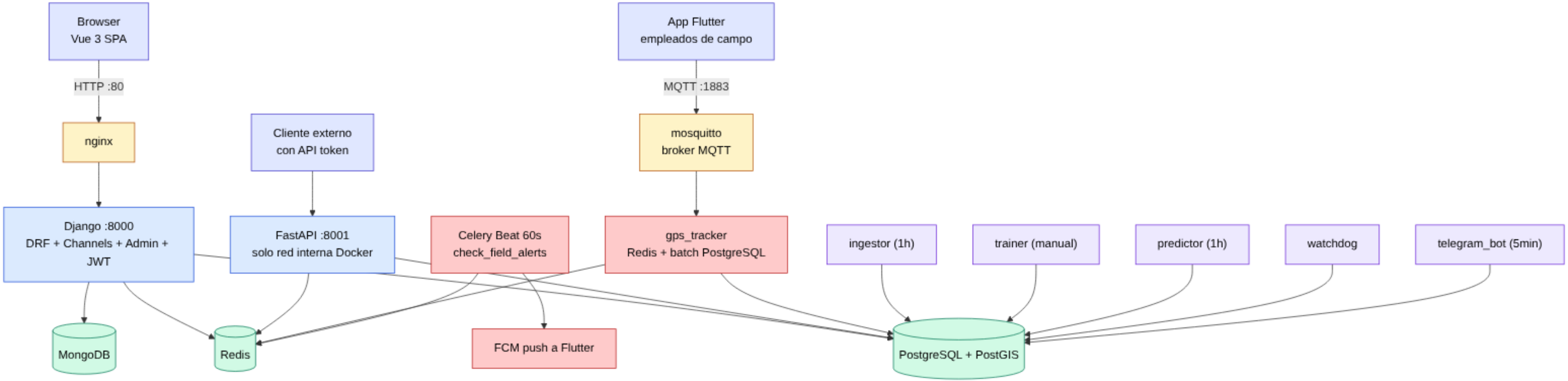
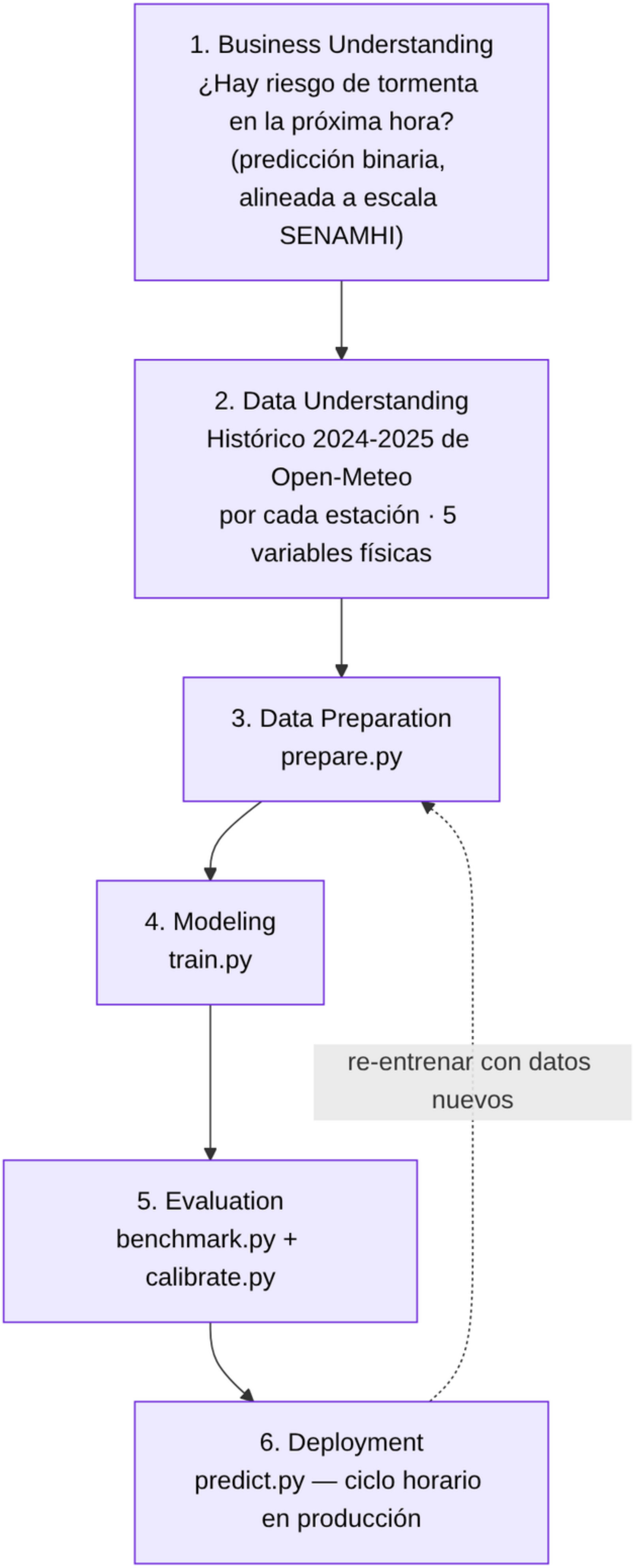
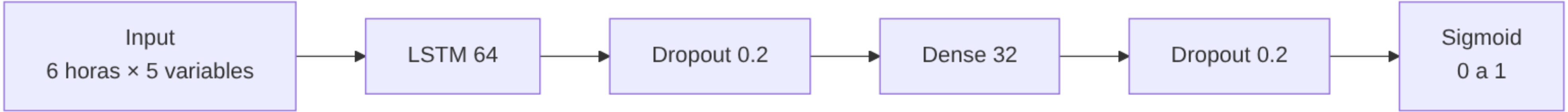


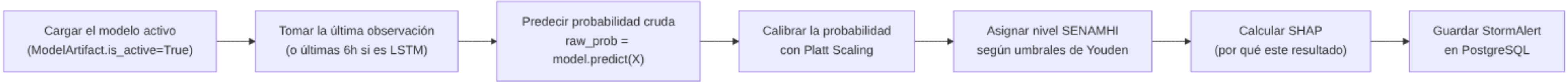
🌐 Open-Meteo API
(datos del clima)
Ingestor
PostgreSQL
+ PostGIS
entrena
Trainer (ML)
activa model
Predictor
(red neuronal MLP/LSTM)
⚡ StormAlert
generada
Django Admin +
Vue SPA (dashboard)
🤖 Telegram Bot
📍 Motor de alertas de
Campo
(distancia
empleado↔estación)
🔔 FCM push
📱 App Flutter
(zumbido + banner)

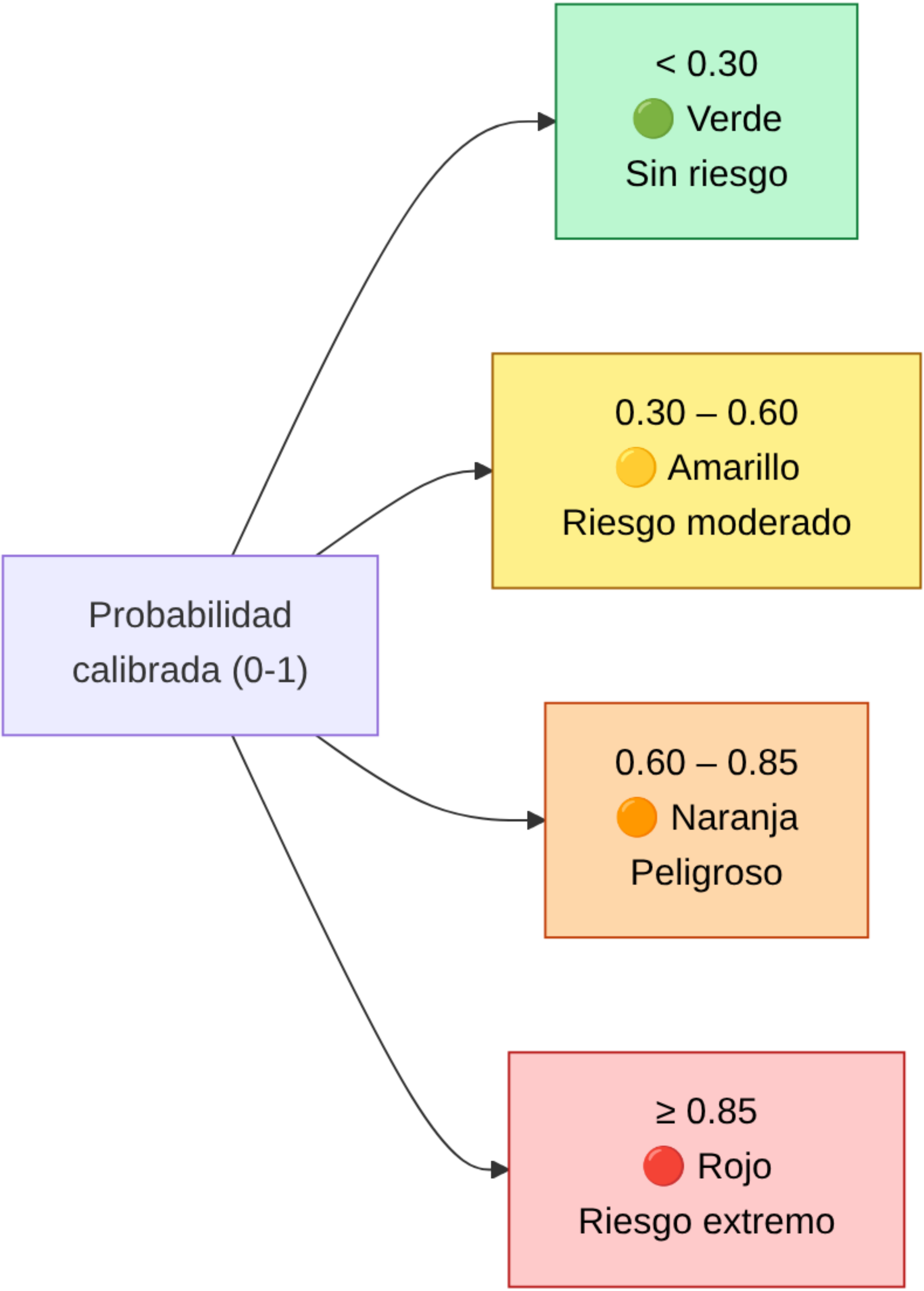

Browser
Vue 3 SPA
HTTP :80
nginx
App Flutter
empleados de campo
MQTT :1883
mosquitto
broker MQTT
Cliente externo
con API token
Django :8000
DRF + Channels + Admin +
JWT
FastAPI :8001
solo red interna Docker
Celery Beat 60s
check_field_alerts
gps_tracker
Redis + batch PostgreSQL
ingestor (1h)
trainer (manual)
predictor (1h)
watchdog
telegram_bot (5min)
MongoDB
Redis
FCM push a Flutter
PostgreSQL + PostGIS


1. Business Understanding
¿Hay riesgo de tormenta en la próxima hora? (predicción binaria, alineada a escala SENAMHI)
2. Data Understanding
Histórico 2024-2025 de Open-Meteo por cada estación · 5 variables físicas
3. Data Preparation
prepare.py
4. Modeling
train.py
5. Evaluation
benchmark.py + calibrate.py
6. Deployment
predict.py — ciclo horario en producción
re-entrenar con datos nuevos

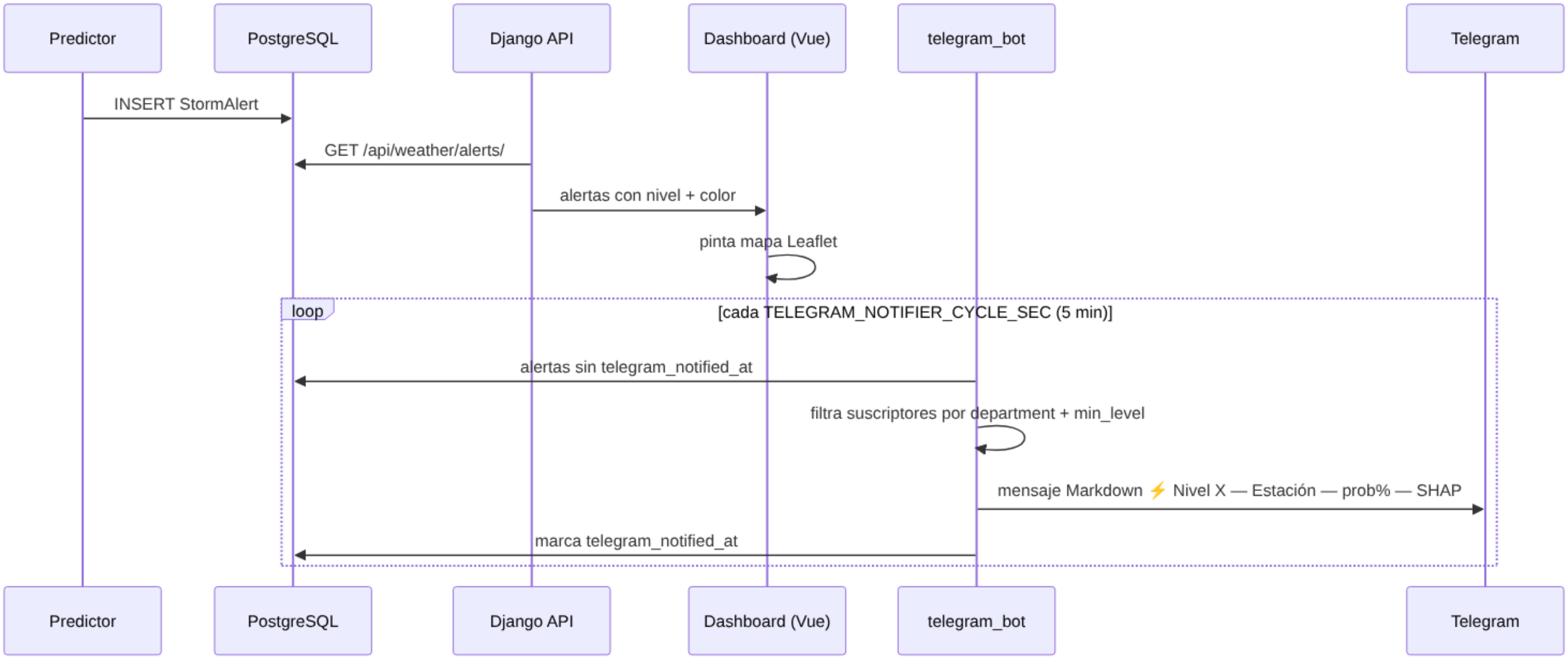

Input
5 variables
Dense 64
Dropout 0.2
Dense 32
Dropout 0.2
Dense 16
Sigmoid
0 a 1

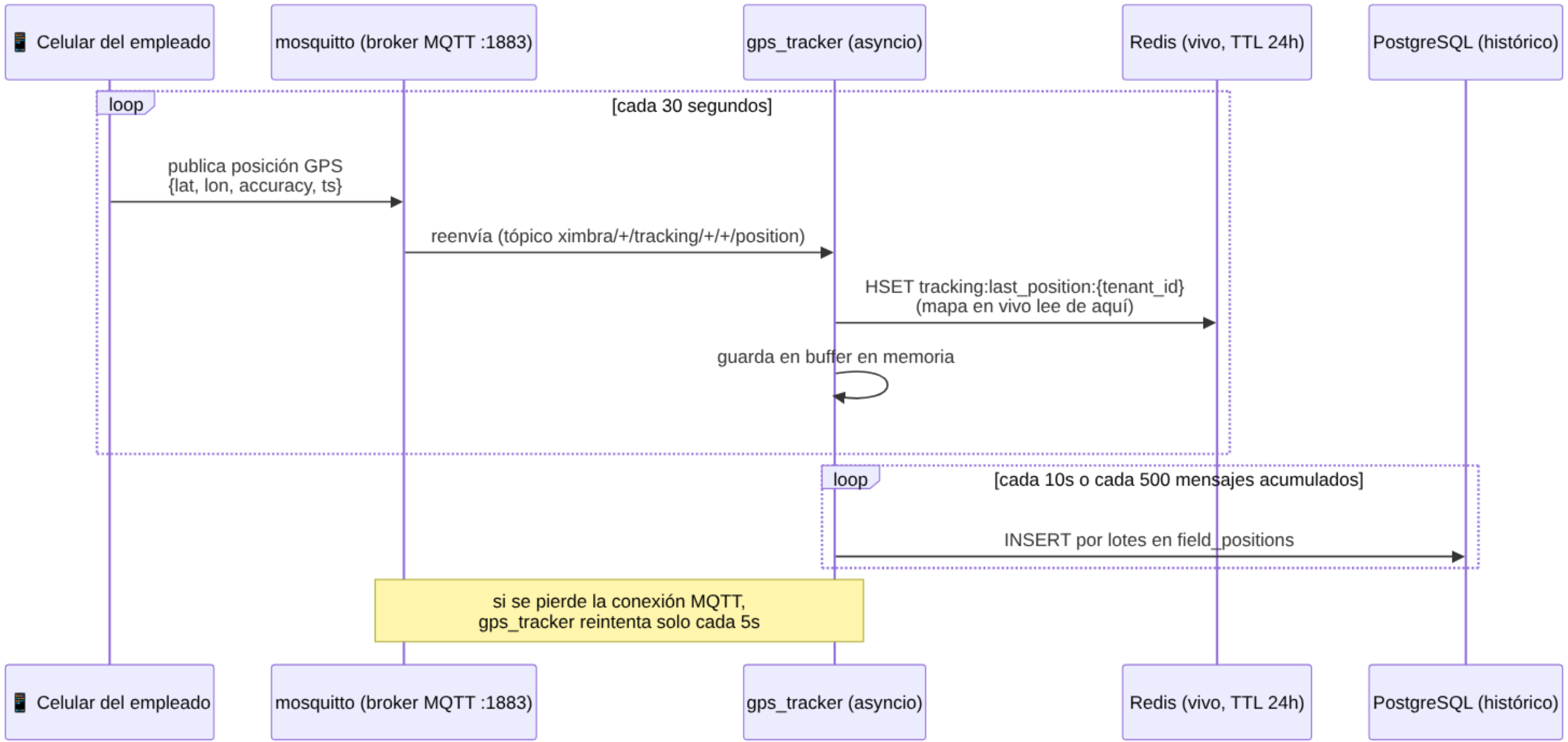

Input
6 horas × 5 variables
LSTM 64
Dropout 0.2
Dense 32
Dropout 0.2
Sigmoid
0 a 1

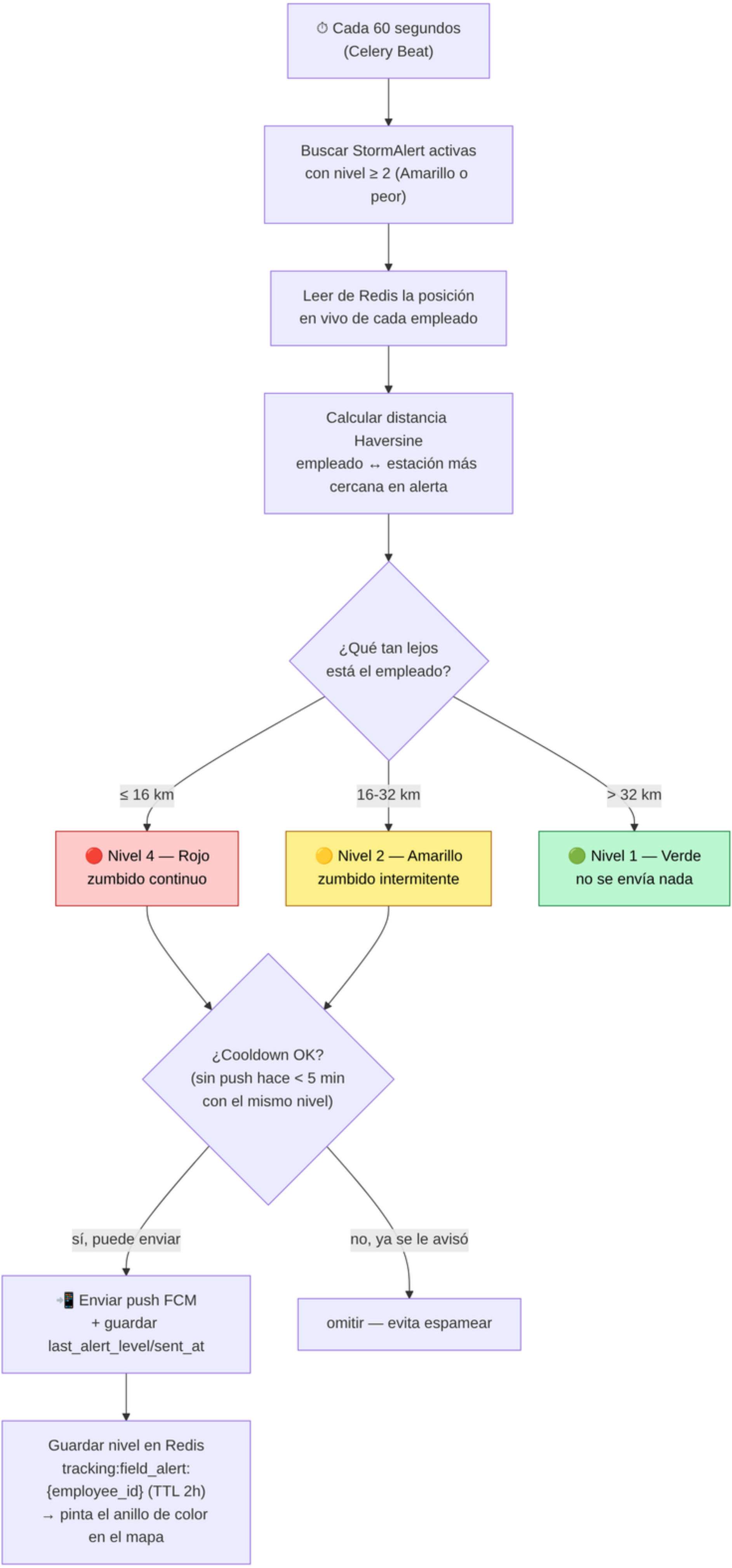

Cargar el modelo activo
(ModelArtifact.is_active=True)
Tomar la última observación
(o últimas 6h si es LSTM)
Predecir probabilidad cruda
raw_prob =
model.predict(X)
Calibrar la probabilidad
con Platt Scaling
Asignar nivel SENAMHI
según umbrales de Youden
Calcular SHAP
(por qué este resultado)
Guardar StormAlert
en PostgreSQL

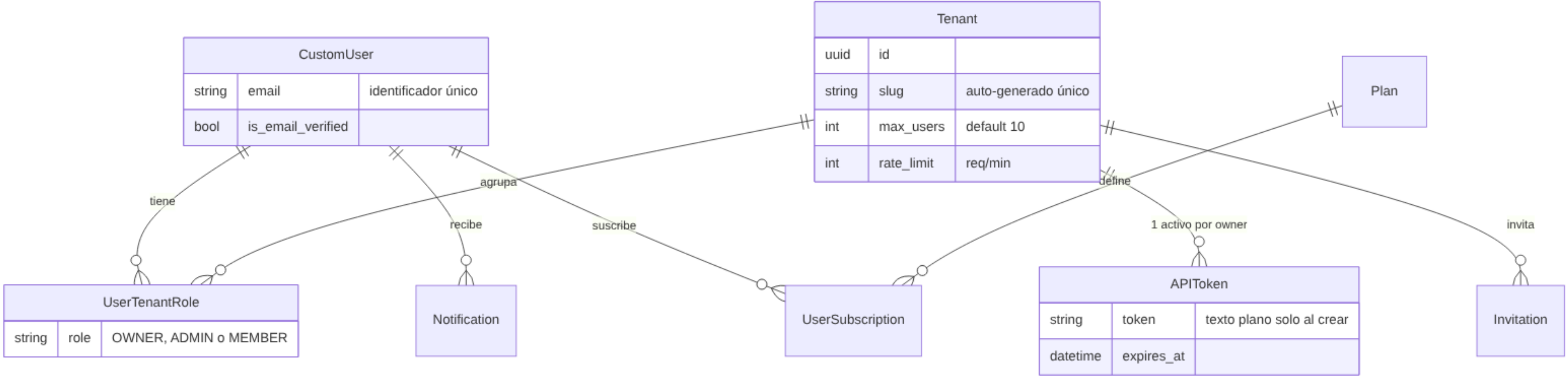


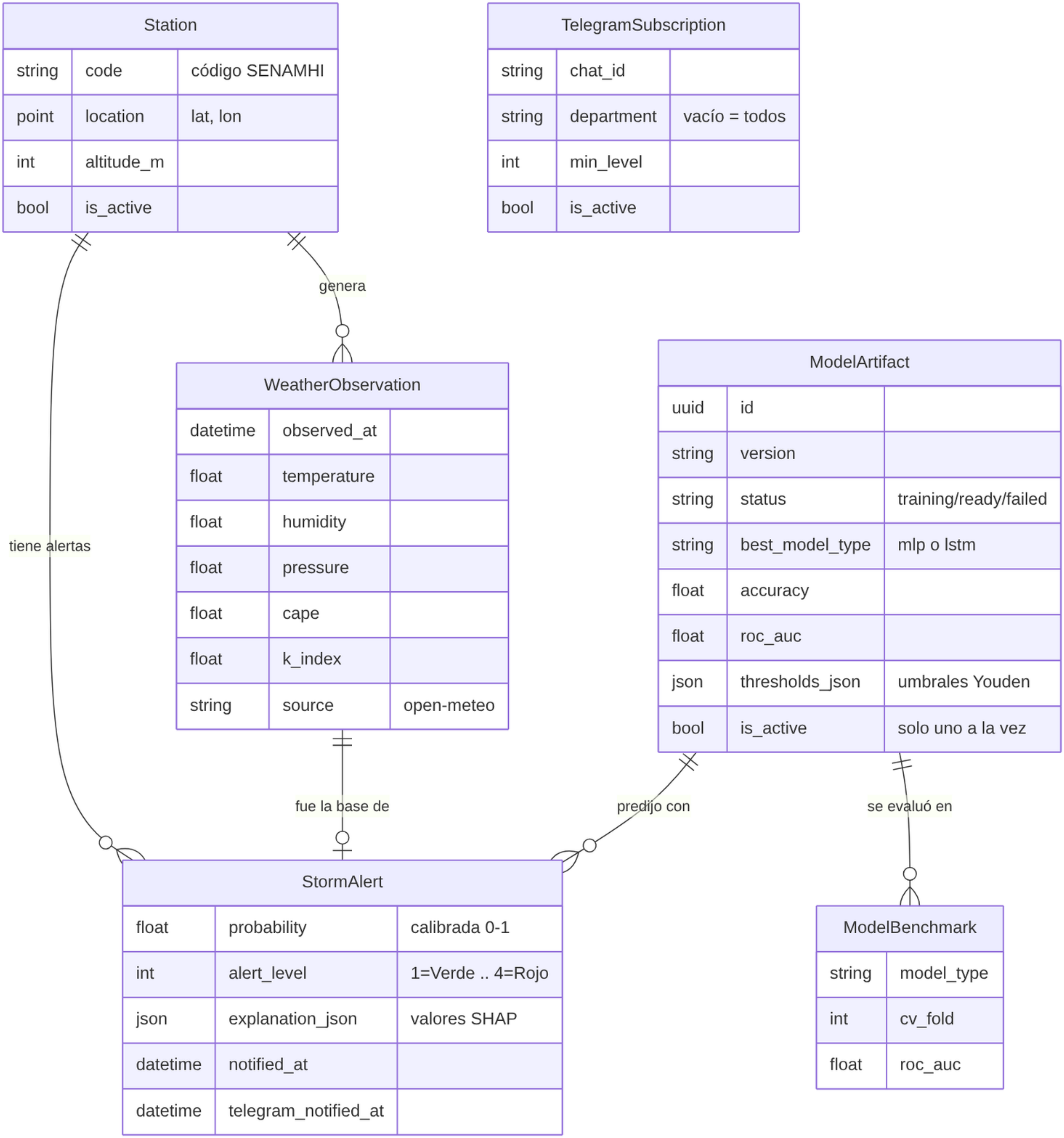
Probabilidad calibrada (0-1)
< 0.30
🟢 Verde
Sin riesgo
0.30 – 0.60
🟡 Amarillo
Riesgo moderado
0.60 – 0.85
🟠 Naranja
Peligroso
≥ 0.85
🔴 Rojo
Riesgo extremo

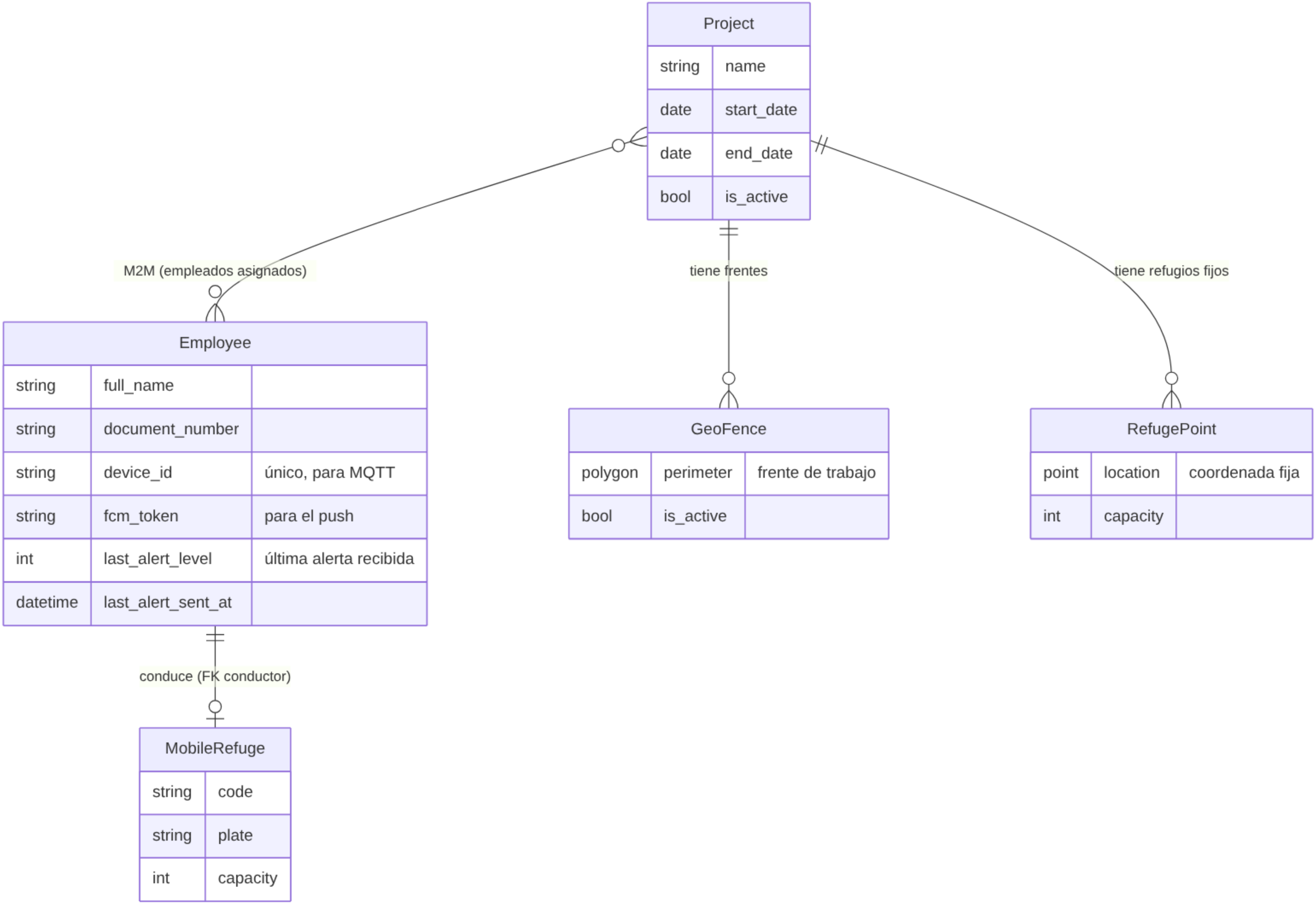

Predictor
PostgreSQL
Django API
Dashboard (Vue)
telegram_bot
Telegram
INSERT StormAlert
GET /api/weather/alerts/
alertas con nivel + color
pinta mapa Leaflet
loop
[cada TELEGRAM_NOTIFIER_CYCLE_SEC (5 min)]
alertas sin telegram_notified_at
filtra suscriptores por department + min_level
mensaje Markdown ⚡ Nivel X — Estación — prob% — SHAP
marca telegram_notified_at

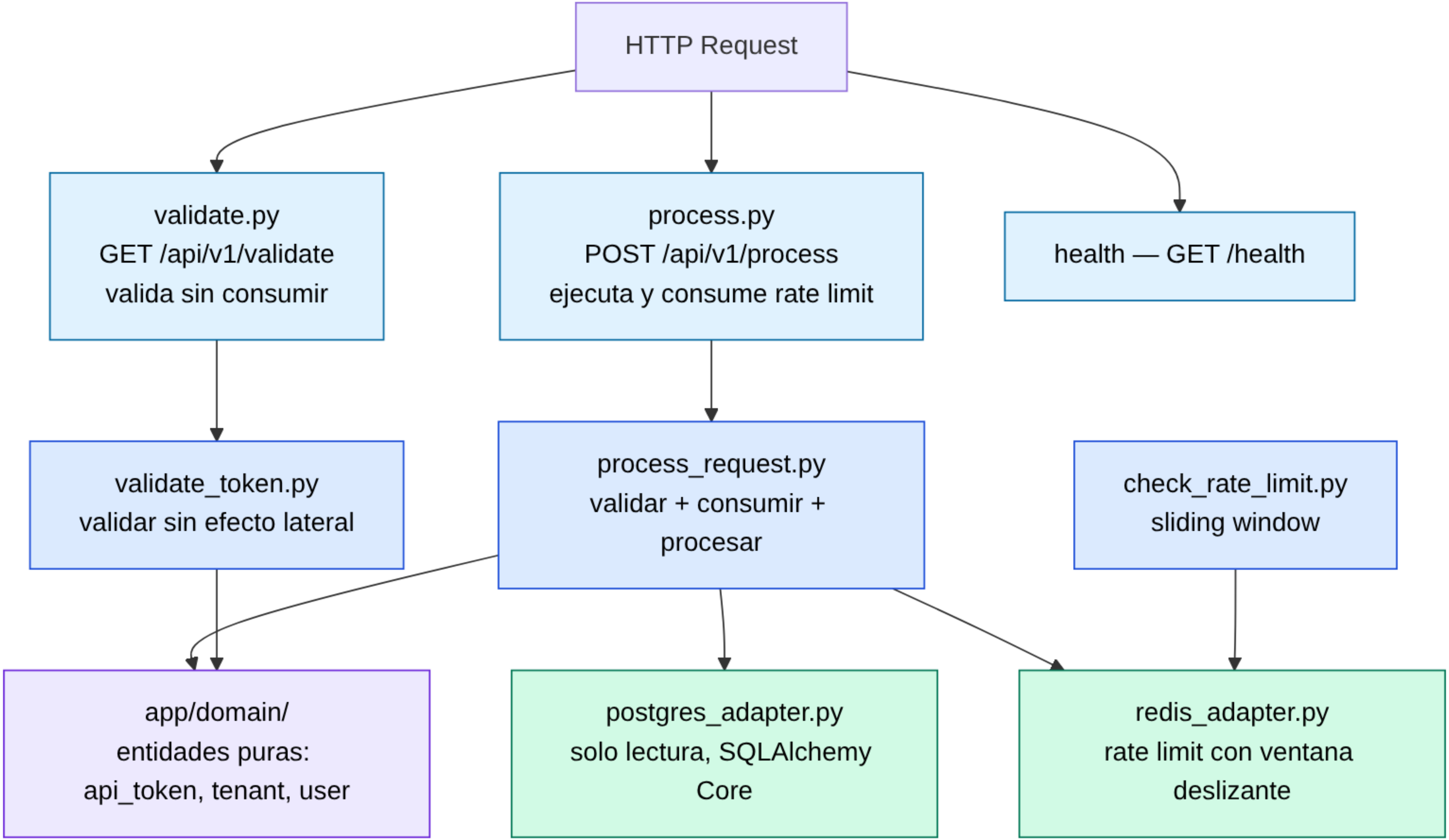

📱 Celular del empleado
mosquitto (broker MQTT :1883)
gps_tracker (asyncio)
Redis (vivo, TTL 24h)
PostgreSQL (histórico)
loop
[cada 30 segundos]
publica posición GPS
{lat, lon, accuracy, ts}
reenvía (tópico ximbra/+/tracking/+/+/position)
HSET tracking:last_position:{tenant_id}
(mapa en vivo lee de aquí)
guarda en buffer en memoria
loop
[cada 10s o cada 500 mensajes acumulados]
INSERT por lotes en field_positions
si se pierde la conexión MQTT,
gps_tracker reintenta solo cada 5s
📱 Celular del empleado
mosquitto (broker MQTT :1883)
gps_tracker (asyncio)
Redis (vivo, TTL 24h)
PostgreSQL (histórico)

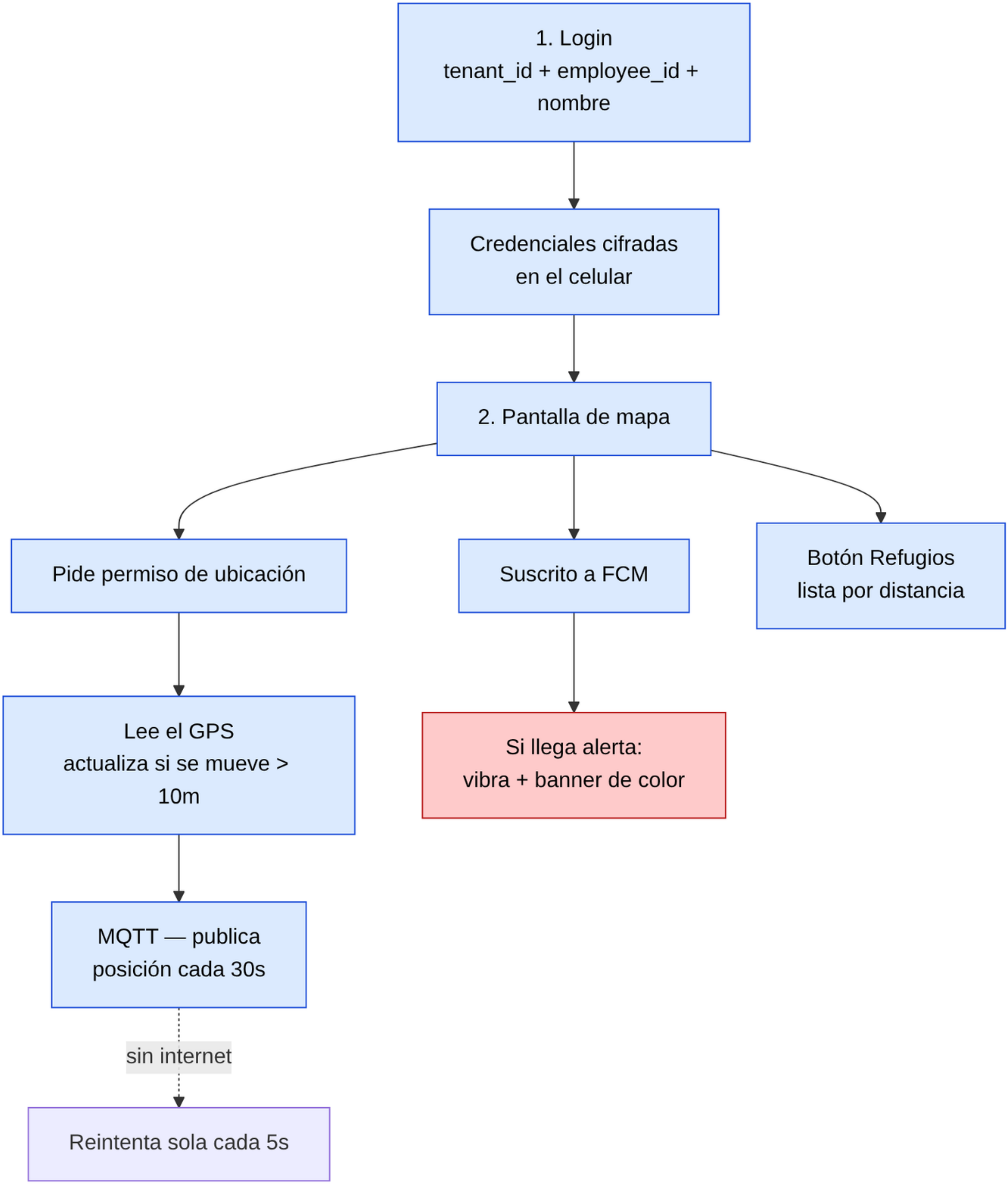

⏱ Cada 60 segundos (Celery Beat)
Buscar StormAlert activas con nivel ≥ 2 (Amarillo o peor)
Leer de Redis la posición en vivo de cada empleado
Calcular distancia Haversine empleado ↔ estación más cercana en alerta
¿Qué tan lejos está el empleado?
≤ 16 km
16-32 km
> 32 km
🔴 Nivel 4 — Rojo zumbido continuo
🟡 Nivel 2 — Amarillo zumbido intermitente
🟢 Nivel 1 — Verde no se envía nada
¿Cooldown OK? (sin push hace < 5 min con el mismo nivel)
sí, puede enviar
no, ya se le avisó
📲 Enviar push FCM + guardar last_alert_level/sent_at
omitir — evita espamear
Guardar nivel en Redis tracking:field_alert:{employee_id} (TTL 2h) → pinta el anillo de color en el mapa

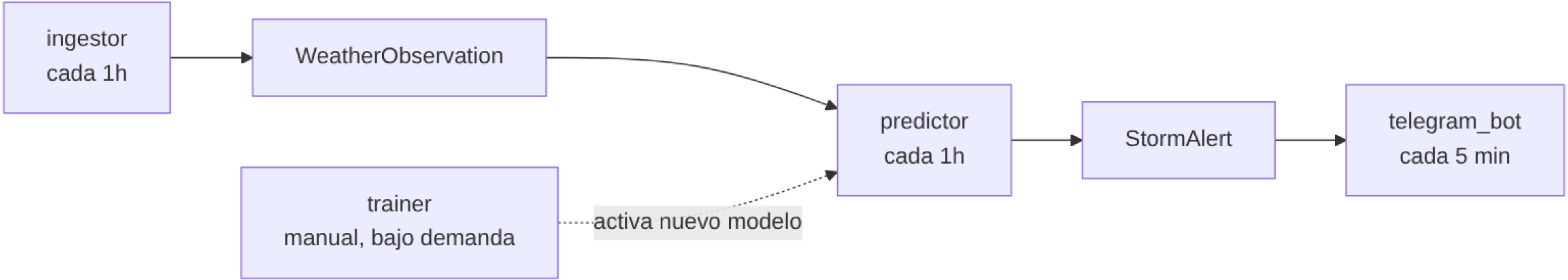

CustomUser
string
email
identificador único
bool
is_email_verified
Tenant
uuid
id
string
slug
auto-generado único
int
max_users
default 10
int
rate_limit
req/min
Plan
tiene
agrupa
recibe
suscribe
define
1 activo por owner
invita
UserTenantRole
string
role
OWNER, ADMIN o MEMBER
Notification
UserSubscription
APIToken
string
token
texto plano solo al crear
datetime
expires_at
Invitation

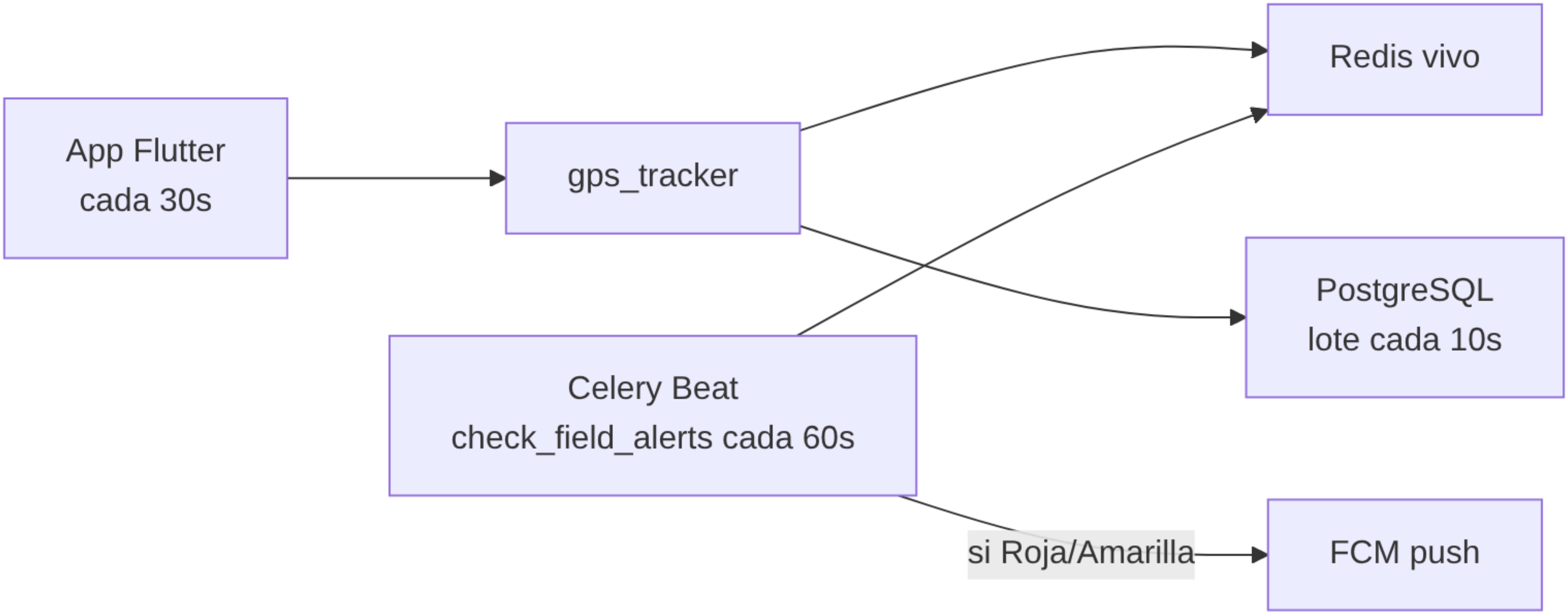

Station
string
code
código SENAMHI
point
location
lat, lon
int
altitude_m
bool
is_active
TelegramSubscription
string
chat_id
string
department
vacío = todos
int
min_level
bool
is_active
genera
tiene alertas
WeatherObservation
datetime
observed_at
float
temperature
float
humidity
float
pressure
float
cape
float
k_index
string
source
open-meteo
ModelArtifact
uuid
id
string
version
string
status
training/ready/failed
string
best_model_type
mlp o lstm
float
accuracy
float
roc_auc
json
thresholds_json
umbrales Youden
bool
is_active
solo uno a la vez
fue la base de
predijo con
se evaluó en
StormAlert
float
probability
calibrada 0-1
int
alert_level
1=Verde .. 4=Rojo
json
explanation_json
valores SHAP
datetime
notified_at
datetime
telegram_notified_at
ModelBenchmark
string
model_type
int
cv_fold
float
roc_auc


Project
string
name
date
start_date
date
end_date
bool
is_active
M2M (empleados asignados)
tiene frentes
tiene refugios fijos
Employee
string
full_name
string
document_number
string
device_id
único, para MQTT
string
fcm_token
para el push
int
last_alert_level
última alerta recibida
datetime
last_alert_sent_at
GeoFence
polygon
perimeter
frente de trabajo
bool
is_active
RefugePoint
point
location
coordenada fija
int
capacity
conduce (FK conductor)
MobileRefuge
string
code
string
plate
int
capacity


HTTP Request
validate.py
GET /api/v1/validate
valida sin consumir
process.py
POST /api/v1/process
ejecuta y consume rate limit
health — GET /health
validate_token.py
validar sin efecto lateral
process_request.py
validar + consumir +
procesar
check_rate_limit.py
sliding window
app/domain/
entidades puras:
api_token, tenant, user
postgres_adapter.py
solo lectura, SQLAlchemy
Core
redis_adapter.py
rate limit con ventana
deslizante


1. Login
tenant_id + employee_id + nombre
Credenciales cifradas en el celular
2. Pantalla de mapa
Pide permiso de ubicación
Suscrito a FCM
Botón Refugios lista por distancia
Lee el GPS actualiza si se mueve > 10m
Si llega alerta: vibra + banner de color
MQTT — publica posición cada 30s
sin internet
Reintenta sola cada 5s


ingestor
cada 1h
WeatherObservation
predictor
cada 1h
StormAlert
telegram_bot
cada 5 min
trainer
manual, bajo demanda
activa nuevo modelo


App Flutter
cada 30s
gps_tracker
Redis vivo
PostgreSQL
lote cada 10s
Celery Beat
check_field_alerts cada 60s
si Roja/Amarilla
FCM push